

Sprawozdanie z zadania: Polimorfizm w C++

1. Jakie klasy zostały zaprojektowane i jakie odpowiedzialności pełni każda z nich?

Zaprojektowano jedną klasę bazową (**Pracownik**) i trzy klasy pochodne (**PracownikEtatowy**, **PracownikGodzinowy**, **PracownikProwizyjny**). Klasa bazowa odpowiada za przechowywanie podstawowych danych identyfikacyjnych i definiuje wspólny interfejs. Klasy pochodne mają za zadanie przechowywanie specyficznych dla danej umowy danych (np. stawka za godzinę) oraz implementację odpowiedniego algorytmu wyliczania pensji.

2. Jakie cechy są wspólne dla wszystkich pracowników, a jakie różnią klasy pochodne?

Cechy wspólne to: imię, nazwisko, identyfikator oraz sam fakt posiadania metody do obliczania pensji i wyświetlania danych. **Różnice** polegają na dodatkowych parametrach (np. etatowy ma stałą kwotę, godzinowy ma stawkę i przepracowane godziny, a prowizyjny sumę sprzedaży i procent) oraz na różnej logice matematycznej wewnątrz metody `obliczPensje()`.

3. Które pola w projekcie zostały ukryte jako `private` i dlaczego?

Wszystkie pola przechowujące dane (imiona, nazwiska, pensje, stawki, godziny) zostały ukryte w sekcjach `private` (lub ew. `protected` dla ułatwienia w klasie bazowej, ale w najlepszej praktyce są w pełni prywatne). Zrobiono to po to, aby zapobiec ich przypadkowej lub celowej zmianie z pominięciem mechanizmów zabezpieczających (hermetyzacja).

4. W jaki sposób program zabezpiecza obiekty przed ustawieniem niepoprawnych danych?

Zabezpieczenie realizowane jest poprzez tzw. settery oraz walidację w konstruktorze. Przed ostatecznym przypisaniem wartości do prywatnego pola, program sprawdza warunkiem logicznym (np. `if (pensja < 0)`), czy wartość jest poprawna. Jeśli nie jest, program rzuca wyjątek (np. `std::invalid_argument`) lub ustawia wartość na domyślną, blokując wprowadzenie ujemnych stawek.

5. Na czym w tym projekcie polega praktyczne zastosowanie hermetyzacji?

Polega na tym, że obiekt z zewnątrz udostępnia użytkownikowi jedynie to, co niezbędne (czyli metody publiczne: podaj pensję, wyświetl dane). Detale przechowywania danych i ich sprawdzania są całkowicie zamknięte "wewnątrz" obiektu. Użytkownik nie musi wiedzieć, jak obiekt trzyma dane, ważne, że działa bezpiecznie.

6. Jakie elementy klasy bazowej zostały odziedziczone przez klasy pochodne?

Klasy pochodne odziedziczyły pola: imię, nazwisko oraz identyfikator pracownika (dostęp do nich zyskują przez konstruktor klasy bazowej i jej metody), a także wirtualne deklaracje metod `obliczPensje()` i `wyswietl()`.

7. Jakie zalety dało użycie dziedziczenia w tym projekcie?

Przede wszystkim wyeliminowano powielanie kodu (tzw. zasada DRY - Don't Repeat Yourself). Każdy pracownik ma imię i nazwisko, więc deklarujemy to tylko raz w klasie bazowej. Ponadto ujednolicono interfejs wszystkich klas.

8. Czy ten program dałoby się napisać bez dziedziczenia? Jeśli tak, to czym różniłoby się takie rozwiązanie?

Tak. Musielibyśmy stworzyć jedną gigantyczną klasę (tzw. "Boską Klasę"), która zawierałaby wszystkie możliwe pola (pensję, stawkę, godziny, sprzedaż) oraz pole typu wyliczeniowego np. `enum TypUmowy`. Metoda `obliczPensje()` musiałaby zawierać wielką instrukcję `switch`. Taki kod byłby jednak fatalny w utrzymaniu i marnowałby dużo pamięci na nieużywane zmienne w konkretnych przypadkach.

9. Które metody zostały zadeklarowane jako wirtualne i dlaczego?

Wirtualne są: `obliczPensje()`, `wyswietl()` oraz destruktor `~Pracownik()`. Zadeklarowano je w ten sposób, aby umożliwić polimorfizm – informując kompilator, że przy wywołaniu tej metody dla wskaźnika klasy bazowej, ma poszukać jej implementacji w odpowiedniej klasie pochodnej, do której faktycznie ten wskaźnik się odnosi.

10. W jaki sposób program wykorzystuje polimorfizm dynamiczny?

Program wykorzystuje wskaźniki typu `Pracownik*` umieszczone w jednej liście (np. `std::vector`). W pętli przechodzi przez wszystkie obiekty, wywołując metodę `obliczPensje()`. Dzięki polimorfizmowi dynamicznemu (tzw. "późnemu wiązaniu"),

aplikacja dopiero w trakcie działania (runtime) decyduje, jaki konkretnie wzór zaaplikować w zależności od faktycznego typu obiektu pod wskaźnikiem.

11. Co się stanie, jeśli metoda `obliczPensje()` nie będzie wirtualna?

Kompilator zastosuje "wczesne wiązanie". Ponieważ wywołujemy metodę z poziomu wskaźnika `Pracownik*`, program uruchomiłby implementację z klasy bazowej, ignorując logikę w klasach pochodnych. (W naszym przypadku, jeśli funkcja jest czysto wirtualna = 0, brak wirtualności i implementacji w bazie spowodowałby po prostu niemożność skompilowania kodu).

12. Jakie są największe zalety podejścia obiektowego w tym zadaniu?

Przede wszystkim skalowalność. Jeśli firma wymyśli nowy rodzaj umowy (np. `PracownikZlecenie`), wystarczy dopisać nową, niewielką klasę dziedziczącą po `Pracownik`. Nie trzeba modyfikować istniejącego kodu (spełnia to zasadę OCP - Open/Closed Principle z SOLID). Kod jest czysty i chroniony przed błędami ludzkimi.

13. Jakie są ograniczenia podejścia obiektowego w tym zadaniu?

Minimalny narzut wydajnościowy związany z tablicą metod wirtualnych (vtable) używaną do obsługi polimorfizmu dynamicznego, oraz konieczność stworzenia sporej ilości wstępnego kodu (tzw. boilerplate) – mnóstwa klas, konstruktorów i setterów dla relatywnie prostej matematyki.